



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка клиент-серверных приложений
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Хамзаев Салахаддин Зелимханович

Группа: ИКБО-11-23

Срок представления к защите: 18.05.2026

Руководитель: ст. преп. Сеницын Анатолий Васильевич

Тема: «Сервис для поиска и бронирования парковочных мест»

Исходные данные: методология двенадцати факторов, Git, Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области для выбранной темы. 2. Выбрать клиент-серверную архитектуру для разрабатываемого приложения и дать её детальное описание с помощью UML. 3. Выбрать программный стек для реализации фуллстек CRUD приложения. 4. Разработать клиентскую и серверную части приложения, реализовать авторизацию и аутентификацию пользователя, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевую модель на некорректные данные. 5. Провести фаззинг-тестирование. 6. Разместить исходный код клиент-серверного приложения в системе контроля версий с наличием Dockerfile и описанием структуры проекта в readme файле. 7. Развернуть клиент-серверное приложение в облаке. 8. Разработать презентацию с графическими материалами.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: [подпись] / Болбаков Р. Г. /, «18» февраля 2026 г.

Задание на КР выдал: [подпись] / Сеницын А. В. /, «18» февраля 2026 г.

Задание на КР получил: [подпись] / Хамзаев С. З. /, «18» февраля 2026 г.



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра игровой индустрии (ИИ)

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: Программная инженерия (09.03.04)

Тема: «Сервис для поиска и бронирования парковочных мест»

Студент: Хамзаев Салахаддин Зелимханович

Группа: ИКБО-11-23

Работа представлена к защите 07.06.2026 / _____ /
(подпись и ф.и.о. студента)

Руководитель: ст.преп., Коваленко Михаил Андреевич

Работа допущена к защите 30.05.2026 (дата) _____ / Синицын А.В.
(подпись и ф.и.о.рук-ля)

Оценка по итогам защиты: _____

_____ 07.06.2026 / Болбаков Роман Геннадьевич, доцент, к.т.н., доцент /

_____ 07.06.2026 / Синицын Анатолий Васильевич, ст.преп. /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

М. РТУ МИРЭА. 2026 г.

РЕФЕРАТ

Отчет 44 с., 20 рис., 4 табл., 17 источн.

ПАРКОВКА, БРОНИРОВАНИЕ, КЛИЕНТ-СЕРВЕРНОЕ ПРИЛОЖЕНИЕ, REST API, GOLANG, REACT, POSTGRESQL, DOCKER, JWT, АУТЕНТИФИКАЦИЯ, РОЛЕВАЯ МОДЕЛЬ, FUZZ-ТЕСТИРОВАНИЕ

Объект исследования – процесс поиска и бронирования парковочных мест в городской среде.

Предмет исследования – клиент-серверное веб-приложение для автоматизации процесса поиска и бронирования парковочных мест.

Цель работы – разработка и апробация fullstack клиент-серверного приложения «ParkEase» для поиска и бронирования парковочных мест с применением методологии двенадцати факторов.

В ходе работы был проведён анализ предметной области, выявлены проблемы существующих процессов поиска парковочных мест, выполнен обзор аналогичных решений. Спроектирована клиент-серверная архитектура с использованием UML. Реализовано fullstack CRUD-приложение с аутентификацией, ролевой моделью, валидацией данных и fuzz-тестированием. Исходный код размещён в системе контроля версий GitHub, приложение контейнеризировано с помощью Docker и развёрнуто в облаке.

Результатом работы является функционирующий сервис «ParkEase», позволяющий водителям находить и бронировать парковочные места, владельцам парковок – управлять своими объектами, а администраторам – контролировать работу всей системы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Описание предметной области	8
1.2 Анализ текущих процессов (AS-IS)	8
1.3 Выявление проблем предметной области	10
1.4 Анализ существующих решений (аналогов)	11
1.5 Определение сущностей и ролей предметной области	13
1.6 Функциональные требования	14
1.7 Нефункциональные требования	16
Выводы по главе 1	16
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	18
2.1 Архитектура системы	18
2.2 Проектирование данных (ER-диаграмма)	20
2.3 Проектирование API	21
2.4 Применение методологии двенадцати факторов	21
2.5 UML-диаграммы	22
3 РЕАЛИЗАЦИЯ	25
3.1 Выбор программного стека	25
3.2 Реализация серверной части	25
3.3 Реализация клиентской части	25
3.4 Аутентификация и авторизация	26
3.5 Проект на удаленном репозитории	26
4 ТЕСТИРОВАНИЕ	27
4.1 Unit-тестирование	27
4.2 Fuzz-тестирование	27
4.3 Тестирование ролевой модели	28
5 РАЗВЁРТЫВАНИЕ	30
5.1 Контейнеризация (Docker)	30
5.2 Развёртывание в облаке	30
5.3 Система контроля версий	33
ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А	38

ПРИЛОЖЕНИЕ Б.....	41
-------------------	----

ВВЕДЕНИЕ

В условиях стремительной урбанизации и роста числа автомобилей в крупных городах проблема поиска свободных парковочных мест становится всё более актуальной. По данным исследований, водители в мегаполисах тратят в среднем от 15 до 30 минут на поиск свободного места для парковки, что приводит к увеличению транспортных заторов, повышенному расходу топлива и негативному воздействию на окружающую среду.

Существующие решения в данной области либо охватывают ограниченное число парковок, либо не предоставляют возможности бронирования, либо имеют высокую стоимость для конечных пользователей. Отсутствие единой информационной системы, объединяющей данные о различных парковках и позволяющей бронировать места заранее, создаёт потребность в разработке нового сервиса.

Целью данной курсовой работы является разработка fullstack клиент-серверного веб-приложения «ParkEase» – сервиса для поиска и бронирования парковочных мест. Приложение должно обеспечивать удобный поиск парковок по местоположению, бронирование мест с выбором времени, управление парковками для их владельцев и администрирование системы.

Для достижения поставленной цели были определены следующие задачи:

- провести анализ предметной области, выявить проблемы и сформулировать требования к системе;
- выполнить обзор и сравнительный анализ существующих решений-аналогов;
- спроектировать клиент-серверную архитектуру приложения с использованием UML;
- выбрать и обосновать программный стек для реализации;
- разработать серверную часть (backend) на языке Go с REST API;
- разработать клиентскую часть (frontend) на React (Next.js);
- реализовать аутентификацию, авторизацию и ролевую модель;

- выполнить тестирование, включая fuzz-тестирование;
- контейнеризировать приложение с помощью Docker и развернуть в облаке;
- применить методологию двенадцати факторов (12-Factor App).

Объектом исследования является процесс поиска и бронирования парковочных мест в городской среде. Предметом исследования – клиент-серверное веб-приложение, автоматизирующее данный процесс.

В работе применяются методы системного анализа, сравнительного анализа, объектно-ориентированного проектирования, а также методология двенадцати факторов для построения современного облачного приложения.

Курсовая работа состоит из введения, пяти глав, заключения, списка использованных источников и приложений. В первой главе проведён анализ предметной области. Во второй главе описано проектирование системы. Третья глава посвящена реализации. В четвёртой главе представлены результаты тестирования. Пятая глава описывает развёртывание приложения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Предметная область данной работы охватывает процессы, связанные с поиском и бронированием парковочных мест в городской среде. Основными участниками процесса являются водители, ищущие место для парковки, владельцы и операторы парковочных комплексов, а также управляющие компании, координирующие работу парковочной инфраструктуры.

В настоящее время автоматизация процесса парковки направлена на решение нескольких ключевых задач: сокращение времени поиска свободного места, обеспечение возможности предварительного бронирования, предоставление актуальной информации о загруженности и тарифах парковок, а также организация удобного способа оплаты.

Система используется в контексте городского транспортного хозяйства и затрагивает как общедоступные муниципальные парковки, так и частные парковочные комплексы – подземные паркинги, многоуровневые гаражи, открытые стоянки при торговых центрах и бизнес-центрах.

Типовые задачи, которые решаются в рамках данной предметной области, включают: ведение реестра парковочных объектов и их характеристик, учёт свободных и занятых мест в режиме реального времени, оформление и отмена бронирований, расчёт стоимости парковки, формирование отчётности для владельцев парковок.

1.2 Анализ текущих процессов (AS-IS)

Для понимания необходимости разработки новой системы рассмотрим, как в настоящее время организован процесс поиска и использования парковочных мест.

Процесс поиска парковки водителем в текущем состоянии выглядит следующим образом. Водитель, приближаясь к месту назначения, начинает визуально искать свободное парковочное место. Он объезжает ближайшие улицы и парковки, пытаясь найти свободное место. Информация о наличии

свободных мест получается исключительно визуально – водитель должен физически проехать мимо парковки, чтобы оценить наличие мест. Забронировать место заранее в большинстве случаев невозможно.

Процесс управления парковкой со стороны владельца также во многом остаётся ручным. Учёт загруженности ведётся визуально или с помощью простейших журналов. Тарифы устанавливаются статически и редко меняются в зависимости от спроса. Статистика по использованию парковки собирается вручную или не собирается вовсе.

Данные о парковках передаются и хранятся децентрализованно. Каждый парковочный оператор ведёт собственный учёт, единой базы данных с информацией о всех доступных парковках не существует. Водитель не имеет возможности из одной точки получить информацию о свободных местах на нескольких парковках одновременно.

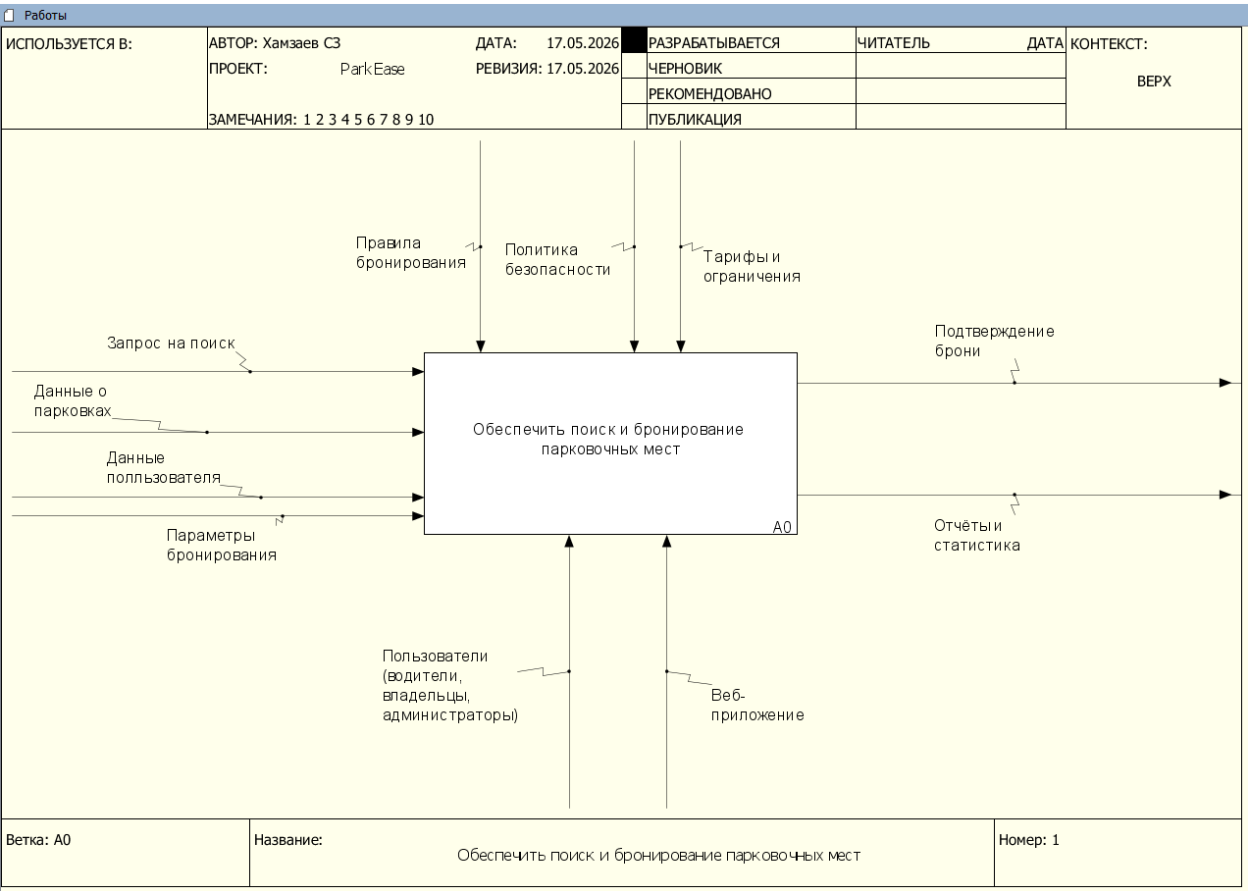


Рисунок 1.1 – Контекстная диаграмма IDEF0 (A-0)

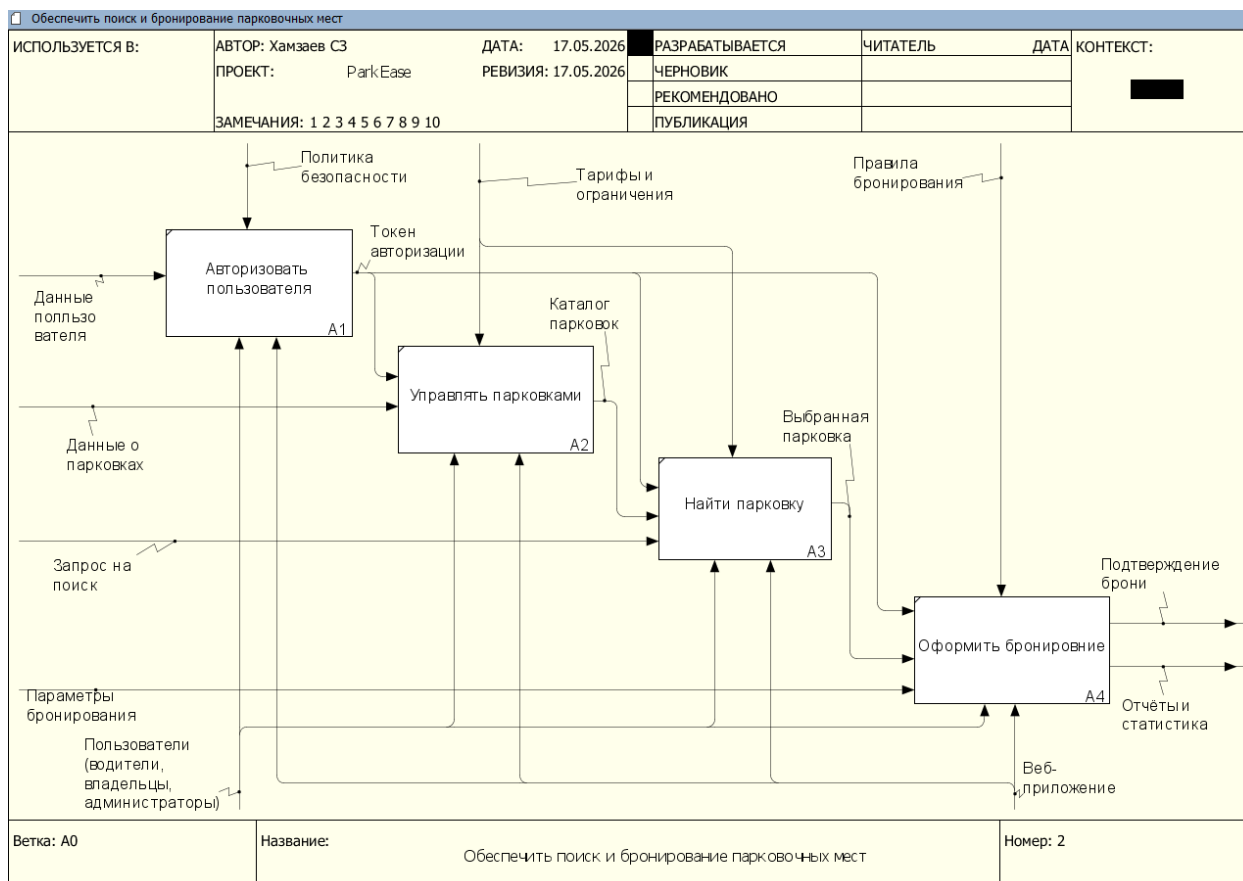


Рисунок 1.2 – Декомпозиция IDEF0 (A0)

1.3 Выявление проблем предметной области

На основе анализа текущего состояния процессов были выявлены следующие проблемы предметной области:

- потеря времени на поиск свободного места – водители тратят значительное количество времени (15–30 минут), объезжая парковки в поисках свободного места, что ведёт к дополнительному расходу топлива и образованию заторов;
- невозможность бронирования – в подавляющем большинстве случаев водитель не может заранее гарантировать наличие свободного места к моменту прибытия;
- отсутствие единой информационной системы – данные о парковках разрознены, нет агрегатора, позволяющего сравнить варианты по расположению, цене и доступности;

– низкая прозрачность загруженности – водитель не имеет актуальной информации о количестве свободных мест на конкретной парковке до момента физического прибытия;

– отсутствие автоматизированного учёта для владельцев – операторы парковок не имеют удобного инструмента для мониторинга загрузки, анализа спроса и управления тарифами;

– нет многопользовательской работы – существующие локальные решения не поддерживают одновременную работу нескольких операторов и не обеспечивают разграничение прав доступа.

Перечисленные проблемы формируют потребность в разработке автоматизированной системы, которая объединит информацию о парковках в единое пространство и позволит водителям бронировать места заранее.

1.4 Анализ существующих решений (аналогов)

Для обоснования необходимости разработки собственной системы был проведён анализ существующих сервисов, решающих схожие задачи.

Яндекс.Парковки – сервис, интегрированный в экосистему Яндекса, предоставляющий информацию о муниципальных платных парковках Москвы. Сервис позволяет просматривать расположение парковок на карте, видеть загруженность и тарифы. К достоинствам относится интеграция с Яндекс.Картами и широкое покрытие муниципальных парковок Москвы. Однако сервис не поддерживает бронирование мест, охватывает только муниципальные парковки (не включает частные), не предоставляет функционал для владельцев парковок и не имеет ролевой модели для различных типов пользователей.

Parkopedia – международный агрегатор парковок, покрывающий более 70 стран мира. Сервис предоставляет информацию о расположении, тарифах и часах работы парковок. Достоинствами являются широкое международное покрытие и мобильные приложения. К недостаткам можно отнести ограниченное покрытие российских городов, отсутствие системы

бронирования для большинства парковок, платную подписку для полного функционала и отсутствие функционала управления парковками для владельцев.

ParkApp – мобильное приложение для поиска парковок, ориентированное на российский рынок. Предоставляет информацию о ближайших парковках с фильтрацией по типу и цене. К достоинствам относятся удобный мобильный интерфейс и покрытие крупных городов России. Недостатки: ограниченная функциональность бронирования, нет веб-версии, нет возможности управления для владельцев парковок, нет административной панели.

Таблица 1.1 – Сравнительный анализ существующих решений

Критерий	Яндекс.Парковки	Parkopedia	ParkApp	ParkEase
Поиск парковок на карте	+	+	+	+
Бронирование мест	–	Частично	Частично	+
Ролевая модель	–	–	–	+
Управление для владельцев	–	–	–	+
Административная панель	–	–	–	+
Отзывы и рейтинги	–	+	+	+
Веб-версия	+	+	–	+
Бесплатный доступ	+	Частично	+	+
Открытый исходный код	–	–	–	+
Покрытие частных парковок	–	+	Частично	+

Как видно из таблицы 1.1, ни одно из рассмотренных решений не предоставляет полного набора функций, необходимого для комплексной автоматизации процесса поиска и бронирования парковочных мест с учётом интересов всех участников – водителей, владельцев парковок и администраторов. Разрабатываемый сервис «ParkEase» призван закрыть эти пробелы, объединив функционал поиска, бронирования, управления парковками и администрирования в едином приложении.

1.5 Определение сущностей и ролей предметной области

На основе проведённого анализа были определены основные сущности предметной области и роли пользователей системы.

Основные сущности системы:

- Пользователь (User) – зарегистрированный участник системы, обладающий определённой ролью и набором прав доступа;
- Парковка (Parking Lot) – парковочный объект с адресом, координатами, описанием и тарифами, принадлежащий определённому владельцу;
- Парковочное место (Parking Spot) – конкретное место на парковке, имеющее номер, тип (стандартное, для инвалидов, электро, VIP) и статус доступности;
- Бронирование (Booking) – факт резервирования конкретного парковочного места пользователем на определённый период времени;
- Отзыв (Review) – оценка и текстовый комментарий пользователя о парковке.

Роли пользователей системы:

- Водитель (user) – основной пользователь системы, осуществляющий поиск парковок, бронирование мест, просмотр истории бронирований и оставление отзывов;
- Владелец парковки (owner) – пользователь, управляющий одной или несколькими парковками, имеющий возможность добавлять и редактировать парковки, просматривать бронирования и статистику;
- Администратор (admin) – пользователь с полным доступом к системе, осуществляющий управление всеми пользователями, модерацию парковок и просмотр системной статистики.

Связи между сущностями: пользователь может владеть несколькими парковками (роль owner); парковка содержит множество парковочных мест; пользователь может создавать бронирования на парковочные места;

пользователь может оставлять отзывы о парковках (не более одного на парковку).

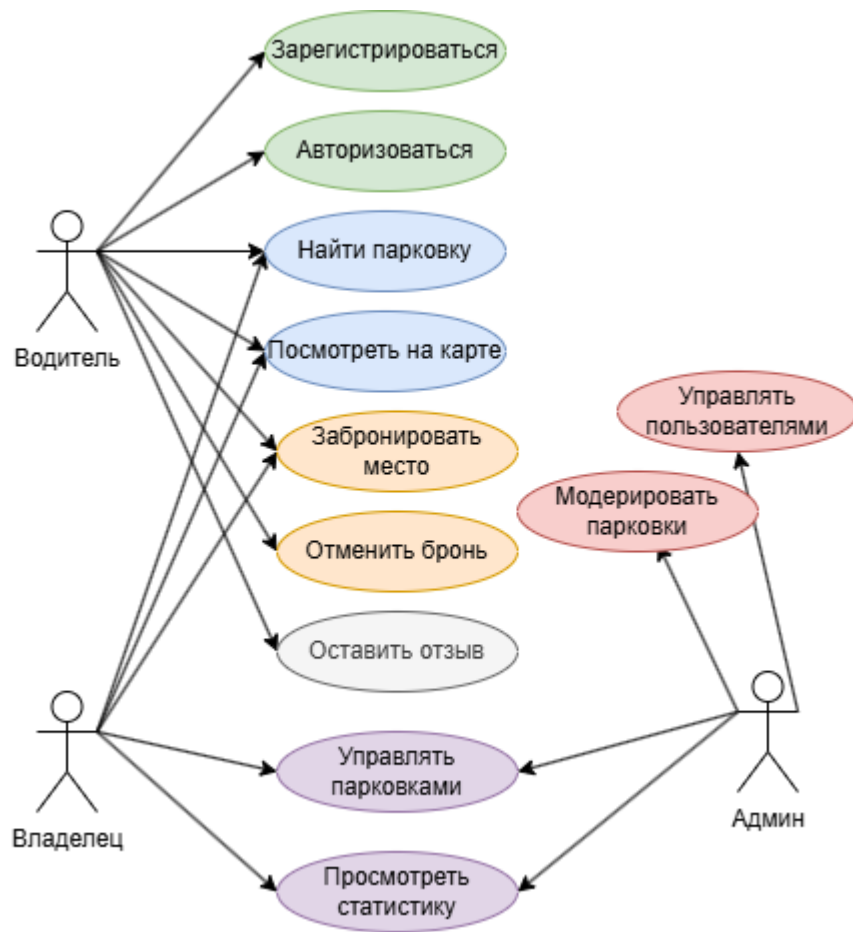


Рисунок 1.3 – Use Case диаграмма системы

1.6 Функциональные требования

На основе выявленных проблем и определённых ролей были сформулированы следующие функциональные требования к системе.

Требования к аутентификации и авторизации:

- система должна обеспечивать регистрацию новых пользователей с указанием email, пароля, имени и телефона;
- система должна обеспечивать вход пользователей с использованием email и пароля;
- система должна использовать JWT-токены для аутентификации запросов к API;

– система должна разграничивать доступ к функционалу на основе роли пользователя.

Требования к функционалу поиска и бронирования:

- система должна позволять осуществлять поиск парковок по местоположению с отображением результатов на интерактивной карте;
- система должна предоставлять детальную информацию о парковке: адрес, описание, тарифы, количество свободных мест;
- система должна позволять бронировать конкретное парковочное место на выбранный период времени;
- система должна проверять отсутствие пересечений бронирований по времени для одного места;
- система должна позволять отменять бронирование;
- система должна автоматически рассчитывать стоимость бронирования на основе тарифа и длительности.

Требования к функционалу управления парковками:

- владелец должен иметь возможность создавать, редактировать и удалять свои парковки;
- владелец должен иметь возможность просматривать бронирования своих парковок;
- владелец должен иметь доступ к статистике использования своих парковок.

Требования к функционалу администрирования:

- администратор должен иметь возможность просматривать и управлять всеми пользователями системы;
- администратор должен иметь возможность модерировать любые парковки;
- администратор должен иметь доступ к общей статистике системы.

Требования к валидации данных:

- система должна валидировать все входные данные и возвращать информативные сообщения об ошибках;

- система должна проверять корректность формата email, номера телефона, номера автомобиля;
- система должна проверять корректность дат бронирования (конец после начала, не в прошлом).

1.7 Нефункциональные требования

Помимо функциональных требований, к системе предъявляются следующие нефункциональные требования:

- производительность – время отклика API не должно превышать 500 мс для основных операций;
- безопасность – пароли пользователей должны храниться в хешированном виде (bcrypt), обмен данными должен осуществляться по протоколу HTTPS, токены доступа должны иметь ограниченный срок действия;
- масштабируемость – архитектура приложения должна быть stateless, что позволяет горизонтальное масштабирование;
- надёжность – приложение должно корректно обрабатывать ошибки и обеспечивать graceful shutdown;
- удобство использования – интерфейс должен быть адаптивным (mobile-friendly) и интуитивно понятным;
- отказоустойчивость – система должна использовать health check эндпоинты для мониторинга состояния;
- переносимость – приложение должно быть контейнеризировано (Docker) для обеспечения единообразной среды развёртывания.

Выводы по главе 1

В результате анализа предметной области было установлено, что процесс поиска и бронирования парковочных мест в настоящее время недостаточно автоматизирован. Основные проблемы связаны с отсутствием единой информационной системы, невозможностью предварительного

бронирования и отсутствием инструментов управления для владельцев парковок.

Сравнительный анализ существующих решений (Яндекс.Парковки, Parkopedia, ParkApp) показал, что ни одно из них не предоставляет полного набора необходимого функционала. Разрабатываемый сервис «ParkEase» призван устранить выявленные недостатки, объединив функции поиска, бронирования, управления и администрирования.

Определены три роли пользователей (водитель, владелец парковки, администратор), пять основных сущностей системы и сформулированы функциональные и нефункциональные требования, которые станут основой для этапа проектирования.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Архитектура системы

Для реализации сервиса была выбрана клиент-серверная архитектура с разделением на Frontend (клиентскую часть) и Backend (серверную часть), взаимодействующие через REST API.

Клиентская часть представляет собой веб-приложение на базе фреймворка Next.js 14 (React) с TypeScript. Для построения пользовательского интерфейса используется библиотека компонентов shadcn/ui и утилитарный CSS-фреймворк Tailwind CSS. Для отображения парковок на карте используется библиотека Leaflet с тайлами OpenStreetMap.

Серверная часть реализована на языке Go (версия 1.22) с использованием легковесного HTTP-маршрутизатора chi. Для работы с базой данных применяется драйвер pgx, обеспечивающий типобезопасное взаимодействие с PostgreSQL без использования тяжёлых ORM. Аутентификация реализована на базе JWT-токенов.

База данных PostgreSQL 16 выбрана как надёжная реляционная СУБД с поддержкой расширений для работы с геоданными. Все компоненты системы контейнеризированы с помощью Docker и оркестрируются через Docker Compose.

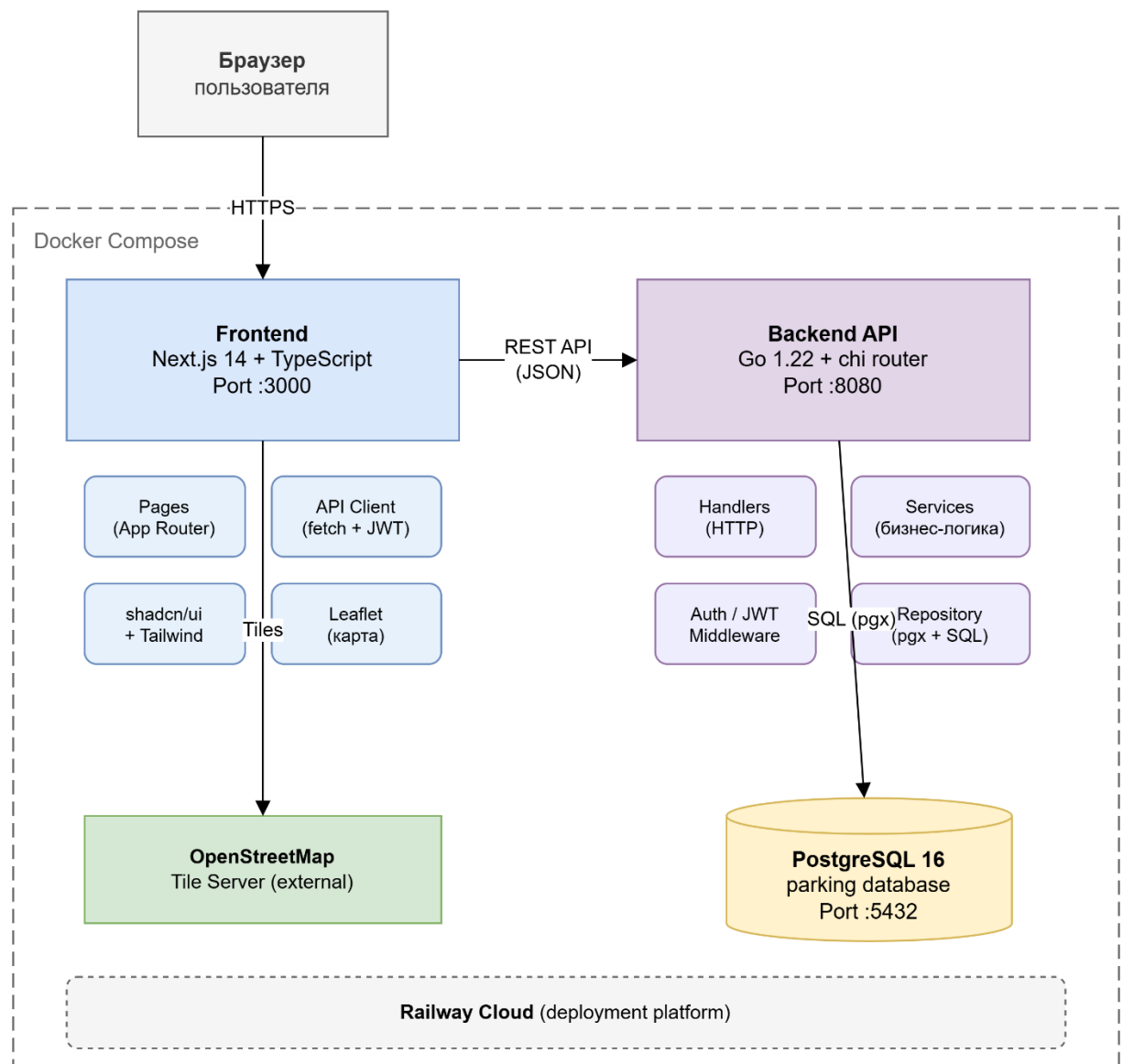


Рисунок 2.1 – Component-диаграмма архитектуры системы

Таблица 2.1 – Обоснование выбора технологий

Компонент	Технология	Обоснование
Backend	Go 1.22 + chi	Высокая производительность, встроенная конкурентность, статическая типизация, компиляция в единый бинарный файл
Frontend	Next.js 14 + TypeScript	SSR для SEO, типизация, широкая экосистема компонентов, App Router для современной архитектуры
БД	PostgreSQL 16	Надёжность, поддержка UUID, JSONB, GIS-расширения, развитые средства индексирования
UI	shadcn/ui + Tailwind	Быстрая разработка, кастомизируемость, адаптивная вёрстка из коробки
Карта	Leaflet + OSM	Бесплатные тайлы, открытый исходный код, без API-ключей

Контейнеризация	Docker	Единообразная среда, воспроизводимые сборки, портативность
Аутентификация	JWT	Stateless, стандарт для REST API, поддержка refresh-токенов

2.2 Проектирование данных (ER-диаграмма)

На основе выявленных сущностей была спроектирована реляционная модель данных, включающая пять основных таблиц: `users`, `parking_lots`, `parking_spots`, `bookings` и `reviews`.

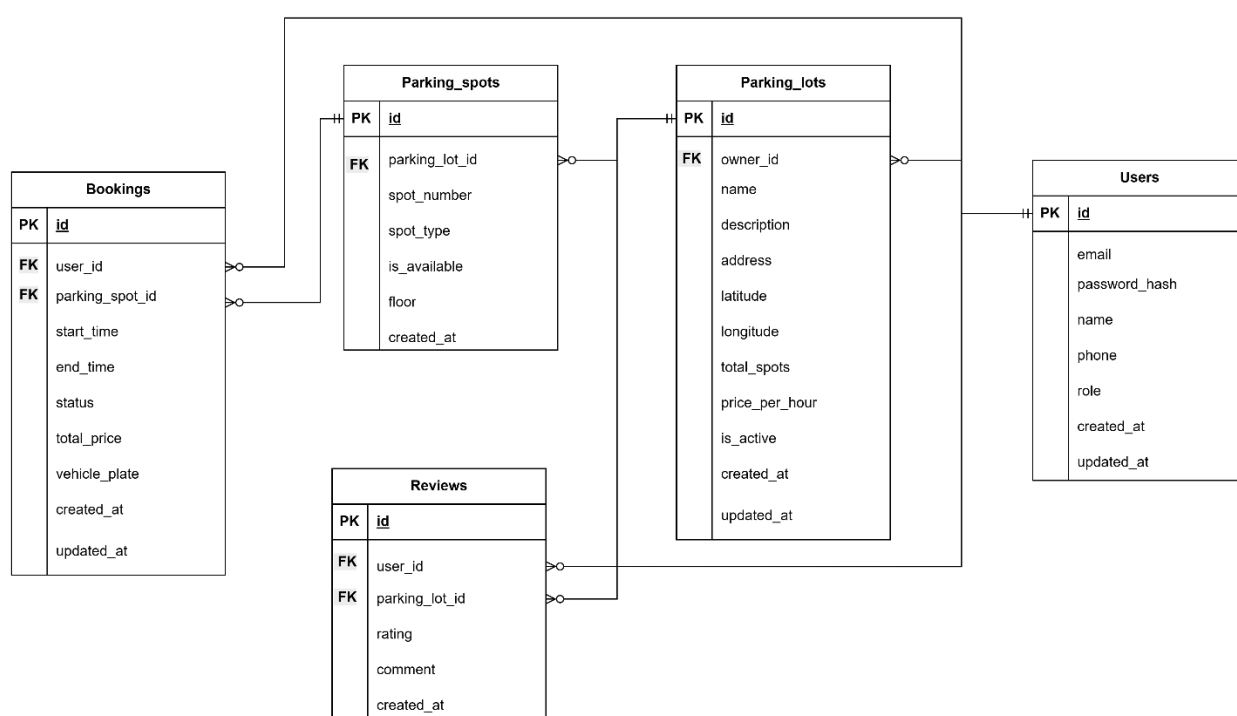


Рисунок 2.2 – ER-диаграмма базы данных

Все первичные ключи используют тип UUID, генерируемый функцией `gen_random_uuid()`. Для хранения временных меток используется тип `TIMESTAMPTZ` (с часовым поясом). Ролевая модель реализована через строковое поле `role` с ограничением `CHECK`. Каждая сущность содержит поля `created_at` и `updated_at` для отслеживания истории изменений.

Между таблицами установлены следующие связи: пользователь (`users`) связан с парковками (`parking_lots`) отношением «один ко многим» через поле `owner_id`; парковка связана с парковочными местами (`parking_spots`)

отношением «один ко многим»; пользователь связан с бронированиями (bookings) отношением «один ко многим»; парковочное место связано с бронированиями отношением «один ко многим»; пользователь и парковка связаны с отзывами (reviews) отношениями «один ко многим» с ограничением уникальности пары (user_id, parking_lot_id).

2.3 Проектирование API

Серверная часть предоставляет REST API, следующее принципам REST-архитектуры: использование HTTP-методов (GET, POST, PUT, DELETE) для соответствующих операций, формат обмена данными JSON, версионирование через URL (/api/v1/), единообразные пути ресурсов.

Таблица 2.2 – Основные эндпоинты REST API

Метод	Путь	Описание	Доступ
POST	/api/v1/auth/register	Регистрация	Публичный
POST	/api/v1/auth/login	Авторизация	Публичный
POST	/api/v1/auth/refresh	Обновление токена	Публичный
GET	/api/v1/users/me	Профиль	Авторизованный
GET	/api/v1/users	Список пользователей	admin
GET	/api/v1/parking-lots	Поиск парковок	Публичный
GET	/api/v1/parking-lots/:id	Детали парковки	Публичный
POST	/api/v1/parking-lots	Создание парковки	owner, admin
PUT	/api/v1/parking-lots/:id	Обновление парковки	owner, admin
DELETE	/api/v1/parking-lots/:id	Удаление парковки	owner, admin
POST	/api/v1/bookings	Создание брони	Авторизованный
GET	/api/v1/bookings	Мои бронирования	Авторизованный
PUT	/api/v1/bookings/:id/cancel	Отмена брони	Авторизованный
POST	/api/v1/parking-lots/:id/reviews	Создание отзыва	user
GET	/api/v1/admin/stats	Статистика	admin

2.4 Применение методологии двенадцати факторов

При проектировании системы была применена методология «The Twelve-Factor App», определяющая принципы построения современных облачных приложений.

Таблица 2.3 – Реализация принципов 12-Factor App

Фактор	Описание	Реализация
I. Codebase	Единая кодовая база	Один репозиторий GitHub
II. Dependencies	Явные зависимости	go.mod, package.json
III. Config	Конфигурация в среде	Переменные окружения (.env)
IV. Backing services	Подключаемые ресурсы	PostgreSQL через DATABASE_URL
V. Build, release, run	Разделение этапов	Multi-stage Docker build
VI. Processes	Stateless-процессы	Состояние хранится в БД
VII. Port binding	Привязка к порту	HTTP-сервер на PORT env
VIII. Concurrency	Масштабирование	Go goroutines
IX. Disposability	Быстрый старт/стоп	Graceful shutdown
X. Dev/prod parity	Паритет сред	Docker Compose
XI. Logs	Логи как потоки	slog → stdout
XII. Admin processes	Админ-задачи	Миграции как отдельная команда

2.5 UML-диаграммы

Для визуализации проектных решений были разработаны UML-диаграммы, описывающие различные аспекты системы.

Диаграмма последовательности (Sequence) описывает процесс бронирования парковочного места – ключевой бизнес-процесс системы. На диаграмме показано взаимодействие четырёх участников: водитель, фронтенд (Next.js), API-сервер (Go) и база данных (PostgreSQL). Процесс включает три основных этапа: поиск парковки, выбор места и оформление бронирования с проверкой пересечений по времени.

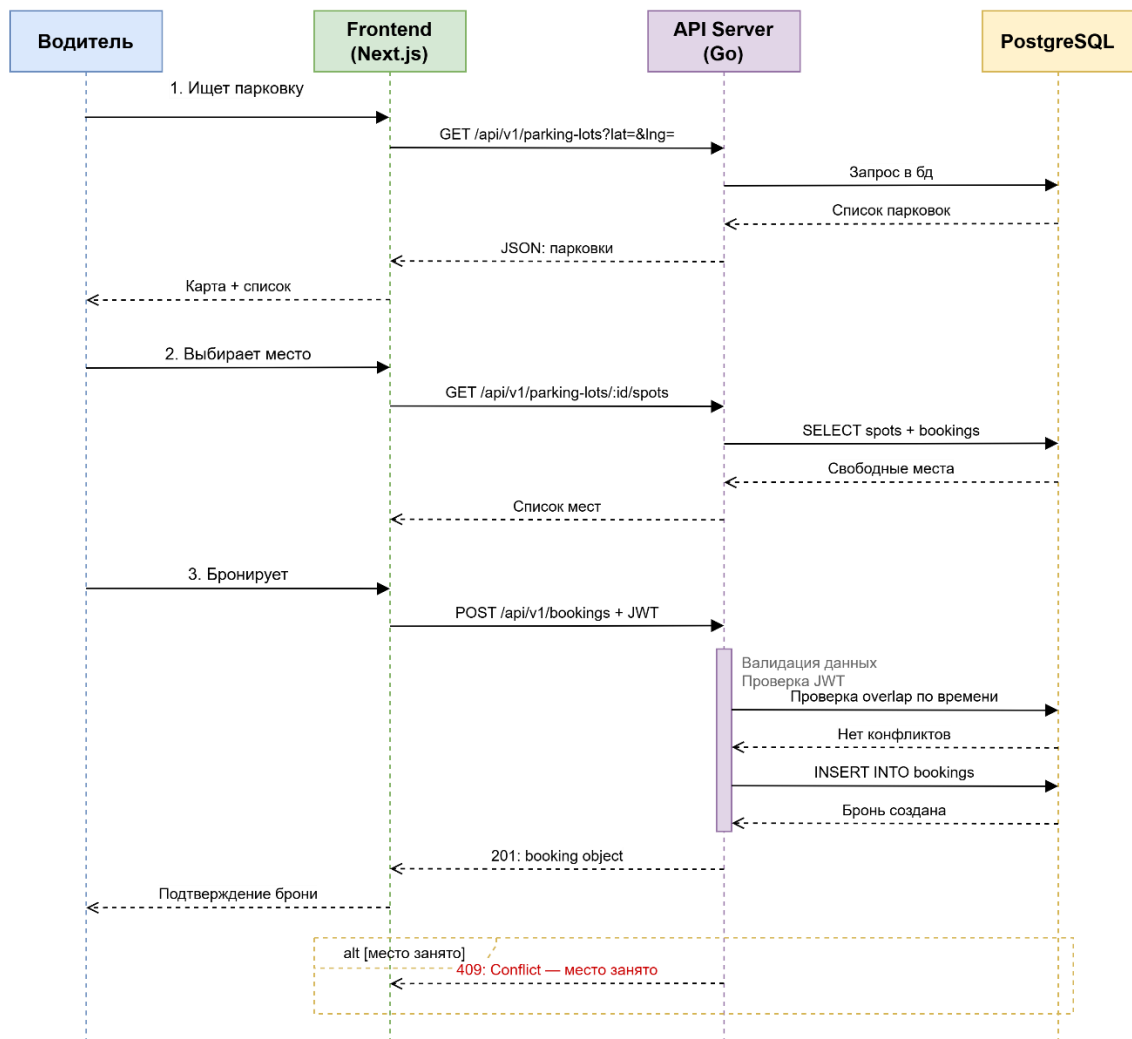


Рисунок 2.3 – Sequence-диаграмма процесса бронирования

Deployment-диаграмма показывает физическое развёртывание системы. Клиентское устройство обращается к фронтенду через HTTPS. Фронтенд и бэкенд развёрнуты как отдельные Docker-контейнеры на платформе Railway. База данных PostgreSQL работает как управляемый сервис. Деплой осуществляется автоматически при push в GitHub.

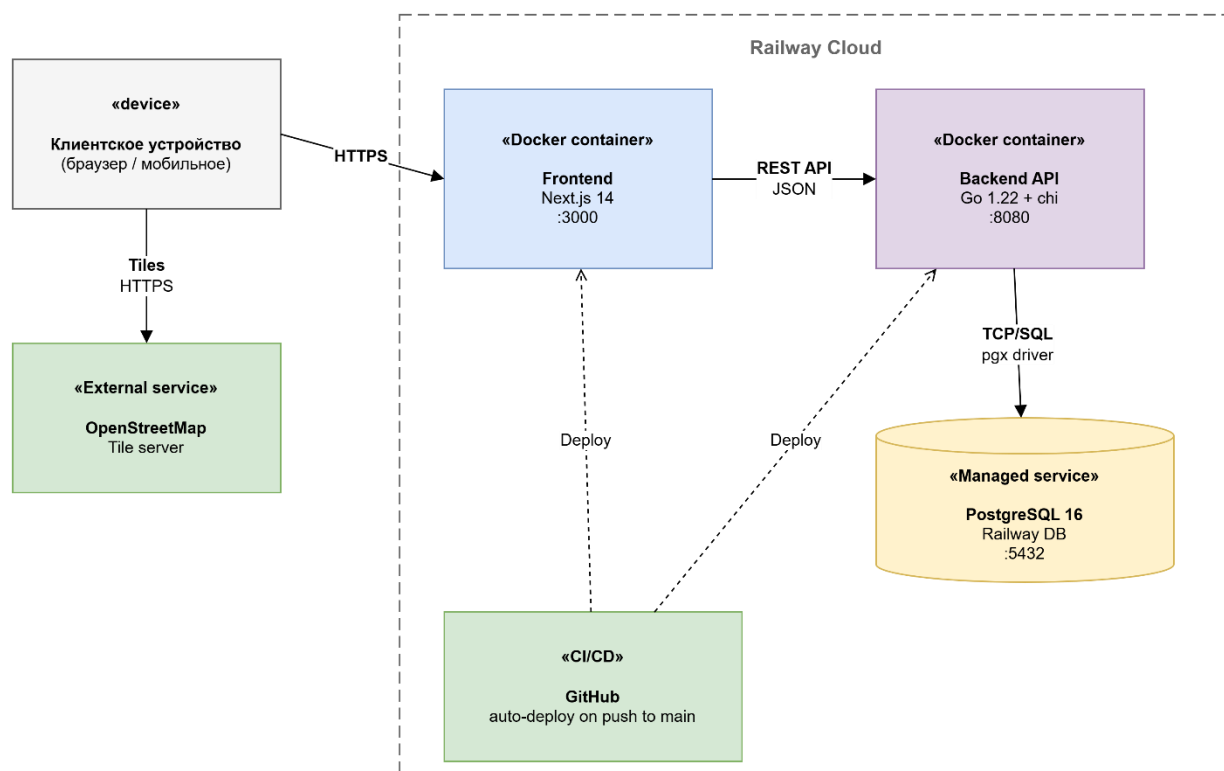


Рисунок 2.4 – Deployment-диаграмма системы

3 РЕАЛИЗАЦИЯ

3.1 Выбор программного стека

На основе результатов проектирования был определён финальный программный стек для реализации приложения. Серверная часть реализована на языке Go версии 1.22 с использованием маршрутизатора chi, который предоставляет минималистичный, но мощный API для построения HTTP-сервисов. Для работы с PostgreSQL используется драйвер pgx, обеспечивающий прямой доступ к базе данных через чистые SQL-запросы без накладных расходов ORM.

Клиентская часть построена на Next.js 14 с App Router – новейшей архитектурой маршрутизации React-приложений, обеспечивающей серверный рендеринг (SSR) и инкрементальную статическую генерацию. TypeScript используется для обеспечения типобезопасности на стороне клиента.

3.2 Реализация серверной части

Серверная часть организована в соответствии с принципами чистой архитектуры (Clean Architecture) и разделена на четыре основных слоя: handler (обработка HTTP-запросов), service (бизнес-логика), repository (работа с базой данных) и model (доменные модели).

Структура проекта следует стандартной компоновке Go-приложений с использованием каталогов cmd/ для точки входа, internal/ для внутренних пакетов и migrations/ для SQL-миграций. Конфигурация загружается из переменных окружения в соответствии с третьим фактором методологии 12-Factor App.

3.3 Реализация клиентской части

Клиентская часть представляет собой одностраничное приложение (SPA) с серверным рендерингом, построенное на Next.js 14. Интерфейс включает следующие основные страницы: главная с интерактивной картой парковок, страница поиска с фильтрами, детальная страница парковки с

информацией о местах и отзывах, форма бронирования, личный кабинет с историей бронирований, панель владельца для управления парковками и административная панель.

Скриншоты клиентской части находятся в приложении Б.

3.4 Аутентификация и авторизация

Аутентификация реализована на основе JWT-токенов с механизмом refresh-токенов. При успешной аутентификации пользователь получает пару токенов: access token (срок действия 15 минут) и refresh token (срок действия 7 дней). Access token используется для аутентификации запросов к защищённым эндпоинтам API. При истечении access token клиент автоматически запрашивает новую пару токенов, используя refresh token.

Пароли пользователей хешируются с использованием алгоритма bcrypt с параметром cost = 10. В базе данных хранится только хеш пароля, что обеспечивает безопасность даже при утечке данных.

Авторизация реализована на уровне middleware. Middleware requireAuth проверяет наличие и валидность JWT-токена. Middleware requireRole принимает список разрешённых ролей и проверяет, что роль текущего пользователя входит в этот список. Для эндпоинтов, где владелец может изменять только свои ресурсы, реализована дополнительная проверка на уровне handler.

3.5 Проект на удаленном репозитории

Репозиторий: <https://github.com/K11a/PRKSP-cursach>

4 ТЕСТИРОВАНИЕ

4.1 Unit-тестирование

Unit-тесты покрывают основную бизнес-логику приложения на уровне сервисного слоя. Тестируются ключевые сценарии: успешное создание бронирования, отклонение бронирования при пересечении по времени, валидация входных данных, генерация и проверка JWT-токенов.

```
github.com/username/parking-service/seeds [no test files]
PS C:\Users\k11a\Desktop\ucheba\3_course\sem2\RKPS\cursach> go test ./... -count=1
? github.com/username/parking-service/cmd/server [no test files]
? github.com/username/parking-service/internal/config [no test files]
ok github.com/username/parking-service/internal/handler 0.751s
? github.com/username/parking-service/internal/migration [no test files]
? github.com/username/parking-service/internal/model [no test files]
? github.com/username/parking-service/internal/repository [no test files]
ok github.com/username/parking-service/internal/service 0.938s
ok github.com/username/parking-service/internal/validator 0.684s
? github.com/username/parking-service/seeds [no test files]
PS C:\Users\k11a\Desktop\ucheba\3_course\sem2\RKPS\cursach> go test ./internal/handler -run=FuzzDecodeJSON -count=1
ok github.com/username/parking-service/internal/handler 0.780s
PS C:\Users\k11a\Desktop\ucheba\3_course\sem2\RKPS\cursach> go test ./internal/validator -run=Fuzz -count=1
ok github.com/username/parking-service/internal/validator 0.579s
PS C:\Users\k11a\Desktop\ucheba\3_course\sem2\RKPS\cursach> |
```

Рисунок 4.1 – Вывод результатов unit-тестов

4.2 Fuzz-тестирование

Fuzz-тестирование (фаззинг) – метод тестирования, при котором на вход программе подаются случайные или полуслучайные данные с целью обнаружения неожиданного поведения, паник и ошибок обработки. В Go фаззинг поддерживается начиная с версии 1.18 через пакет testing.

В рамках данной работы fuzz-тесты были написаны для следующих компонентов:

- валидация email – проверка, что функция ValidateEmail не вызывает панику на произвольных входных строках;
- валидация номера телефона – проверка корректной обработки нестандартных форматов;
- валидация номера автомобиля – проверка обработки произвольных строк;
- парсинг дат бронирования – проверка, что парсинг не вызывает панику на невалидных датах;
- обработка JSON body – проверка, что десериализация произвольного JSON не приводит к панике.

```

ok      github.com/username/parking-service/internal/service    0.963s
=== RUN   TestValidateBookingTime
=== RUN   TestValidateBookingTime/valid_future_range
=== RUN   TestValidateBookingTime/start_in_past
=== RUN   TestValidateBookingTime/end_before_start
--- PASS: TestValidateBookingTime (0.00s)
    --- PASS: TestValidateBookingTime/valid_future_range (0.00s)
    --- PASS: TestValidateBookingTime/start_in_past (0.00s)
    --- PASS: TestValidateBookingTime/end_before_start (0.00s)
=== RUN   FuzzValidateEmail
=== RUN   FuzzValidateEmail/seed#0
=== RUN   FuzzValidateEmail/seed#1
=== RUN   FuzzValidateEmail/seed#2
--- PASS: FuzzValidateEmail (0.00s)
    --- PASS: FuzzValidateEmail/seed#0 (0.00s)
    --- PASS: FuzzValidateEmail/seed#1 (0.00s)
    --- PASS: FuzzValidateEmail/seed#2 (0.00s)
=== RUN   FuzzValidatePhone
=== RUN   FuzzValidatePhone/seed#0
=== RUN   FuzzValidatePhone/seed#1
=== RUN   FuzzValidatePhone/seed#2
--- PASS: FuzzValidatePhone (0.00s)
    --- PASS: FuzzValidatePhone/seed#0 (0.00s)
    --- PASS: FuzzValidatePhone/seed#1 (0.00s)
    --- PASS: FuzzValidatePhone/seed#2 (0.00s)
=== RUN   FuzzValidateVehiclePlate
=== RUN   FuzzValidateVehiclePlate/seed#0
=== RUN   FuzzValidateVehiclePlate/seed#1
=== RUN   FuzzValidateVehiclePlate/seed#2
--- PASS: FuzzValidateVehiclePlate (0.00s)
    --- PASS: FuzzValidateVehiclePlate/seed#0 (0.00s)
    --- PASS: FuzzValidateVehiclePlate/seed#1 (0.00s)
    --- PASS: FuzzValidateVehiclePlate/seed#2 (0.00s)
=== RUN   FuzzParseBookingDate
=== RUN   FuzzParseBookingDate/seed#0
=== RUN   FuzzParseBookingDate/seed#1
=== RUN   FuzzParseBookingDate/seed#2
--- PASS: FuzzParseBookingDate (0.00s)
    --- PASS: FuzzParseBookingDate/seed#0 (0.00s)
    --- PASS: FuzzParseBookingDate/seed#1 (0.00s)
    --- PASS: FuzzParseBookingDate/seed#2 (0.00s)
PASS

```

Рисунок 4.2 – Вывод результатов fuzz-тестов

4.3 Тестирование ролевой модели

Для подтверждения корректности работы ролевой модели были разработаны интеграционные тесты, проверяющие разграничение доступа:

- пользователь с ролью user пытается создать парковку – ожидается HTTP 403 Forbidden;
- пользователь с ролью user пытается удалить чужое бронирование – ожидается HTTP 403 Forbidden;

- пользователь с ролью owner пытается получить список всех пользователей – ожидается HTTP 403 Forbidden;
- запрос с невалидным JWT-токеном – ожидается HTTP 401 Unauthorized;
- запрос с истёкшим JWT-токеном – ожидается HTTP 401 Unauthorized.

```
go test ./... -v -count=1
?      github.com/username/parking-service/cmd/server [no test files]
?      github.com/username/parking-service/internal/config [no test files]
=== RUN   TestRequireAuthRejectsInvalidJWT
--- PASS: TestRequireAuthRejectsInvalidJWT (0.00s)
=== RUN   TestRequireRoleRejectsWrongRole
--- PASS: TestRequireRoleRejectsWrongRole (0.00s)
=== RUN   TestRequireAuthAcceptsValidJWT
--- PASS: TestRequireAuthAcceptsValidJWT (0.00s)
=== RUN   FuzzDecodeJSON
=== RUN   FuzzDecodeJSON/seed#0
=== RUN   FuzzDecodeJSON/seed#1
=== RUN   FuzzDecodeJSON/seed#2
--- PASS: FuzzDecodeJSON (0.00s)
    --- PASS: FuzzDecodeJSON/seed#0 (0.00s)
    --- PASS: FuzzDecodeJSON/seed#1 (0.00s)
    --- PASS: FuzzDecodeJSON/seed#2 (0.00s)
PASS
```

Рисунок 4.3 – Вывод результатов fuzz-тестов

5 РАЗВЁРТЫВАНИЕ

5.1 Контейнеризация (Docker)

Для обеспечения переносимости и воспроизводимости среды выполнения приложение контейнеризировано с использованием Docker. Для серверной части используется multi-stage сборка: на первом этапе (builder) компилируется Go-приложение, на втором этапе минимальный образ Alpine Linux копирует только скомпилированный бинарный файл. Это позволяет получить итоговый образ размером менее 30 МБ.

Docker Compose используется для оркестрации трёх сервисов: PostgreSQL (база данных), Backend (Go API) и Frontend (Next.js). Для PostgreSQL настроен health check, гарантирующий, что бэкенд запустится только после готовности базы данных.

В приложении А находятся листинги файлов Docker.

```
PS C:\Users\kila\Desktop\ucheba\3_course\sem2\RKPS\cursach> docker-compose up --build -d
=> CACHED [frontend runner 4/5] COPY --from=builder /app/.next/standalone ./ 0.0s
=> CACHED [frontend runner 5/5] COPY --from=builder /app/.next/static ./next/static 0.0s
=> [frontend] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:6c1866c5ee44921bf58c86089c7e9bf20f3223ff32a89fdd3f79383a1518ea0a 0.0s
=> => exporting config sha256:875f5aaa4a3ca06ef5b11bfe4be1b9ed736cedf57afdd0263b70a451d070e85b 0.0s
=> => exporting attestation manifest sha256:9096f75011ce54b6b06d36a7b3111d5fa0cc07906150419228c9 0.1s
=> => exporting manifest list sha256:6d6b1bbf4c416404edb7d72f8400353801482539e38d0beef1c396402bc 0.0s
=> => naming to docker.io/library/cursach-frontend:latest 0.0s
=> => unpacking to docker.io/library/cursach-frontend:latest 0.0s
=> [frontend] resolving provenance for metadata file 0.0s
[+] Running 6/6
✓ backend Built 0.0s
✓ frontend Built 0.0s
✓ Network cursach_default Created 0.3s
✓ Container cursach-postgres-1 Healthy 6.1s
✓ Container cursach-backend-1 Started 6.3s
✓ Container cursach-frontend-1 Started 6.5s
PS C:\Users\kila\Desktop\ucheba\3_course\sem2\RKPS\cursach>
```

Рисунок 5.1 – Вывод сборки docker-compose

5.2 Развёртывание в облаке

Для развёртывания приложения в облаке была выбрана платформа Railway, предоставляющая бесплатный tier для хостинга приложений с поддержкой Docker и управляемыми базами данных PostgreSQL. Деплой осуществляется автоматически при push в ветку main репозитория GitHub.

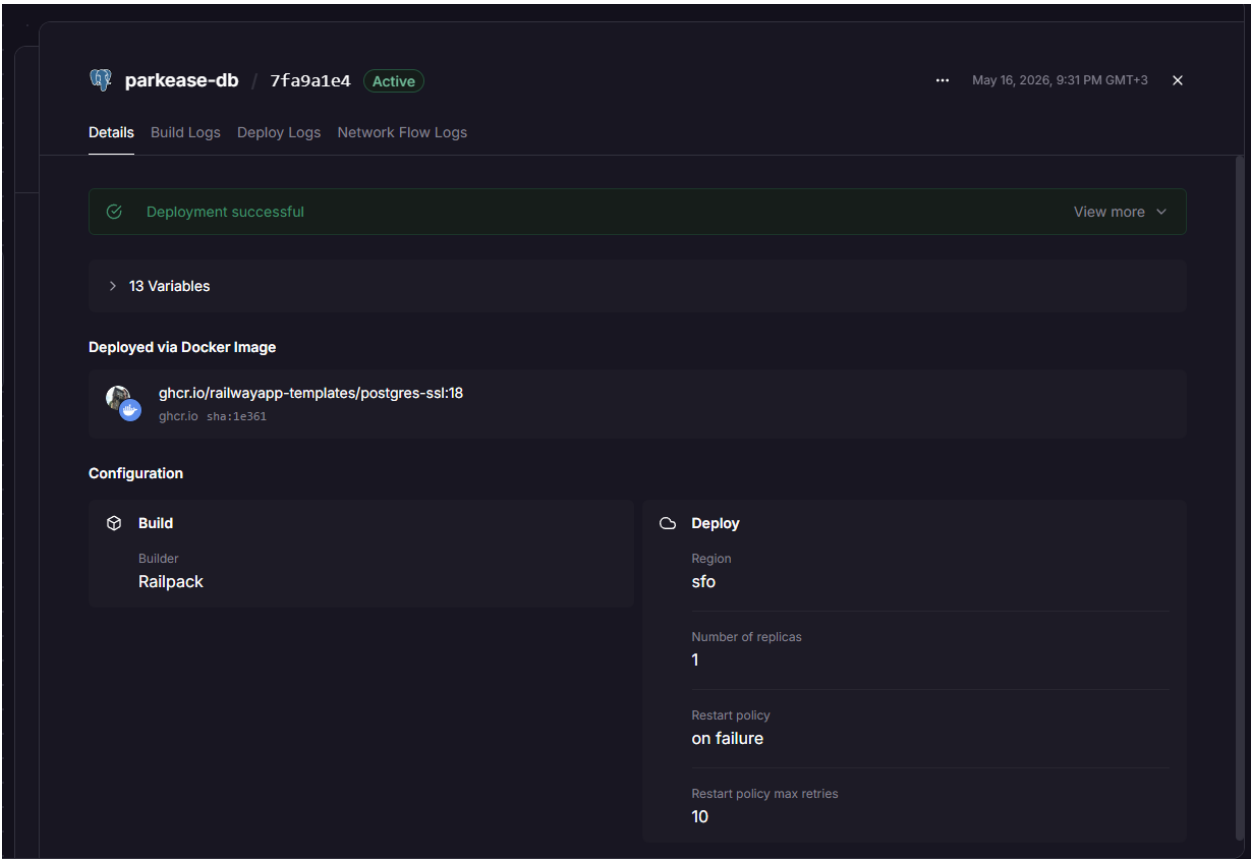


Рисунок 5.4 –База данных Postgres

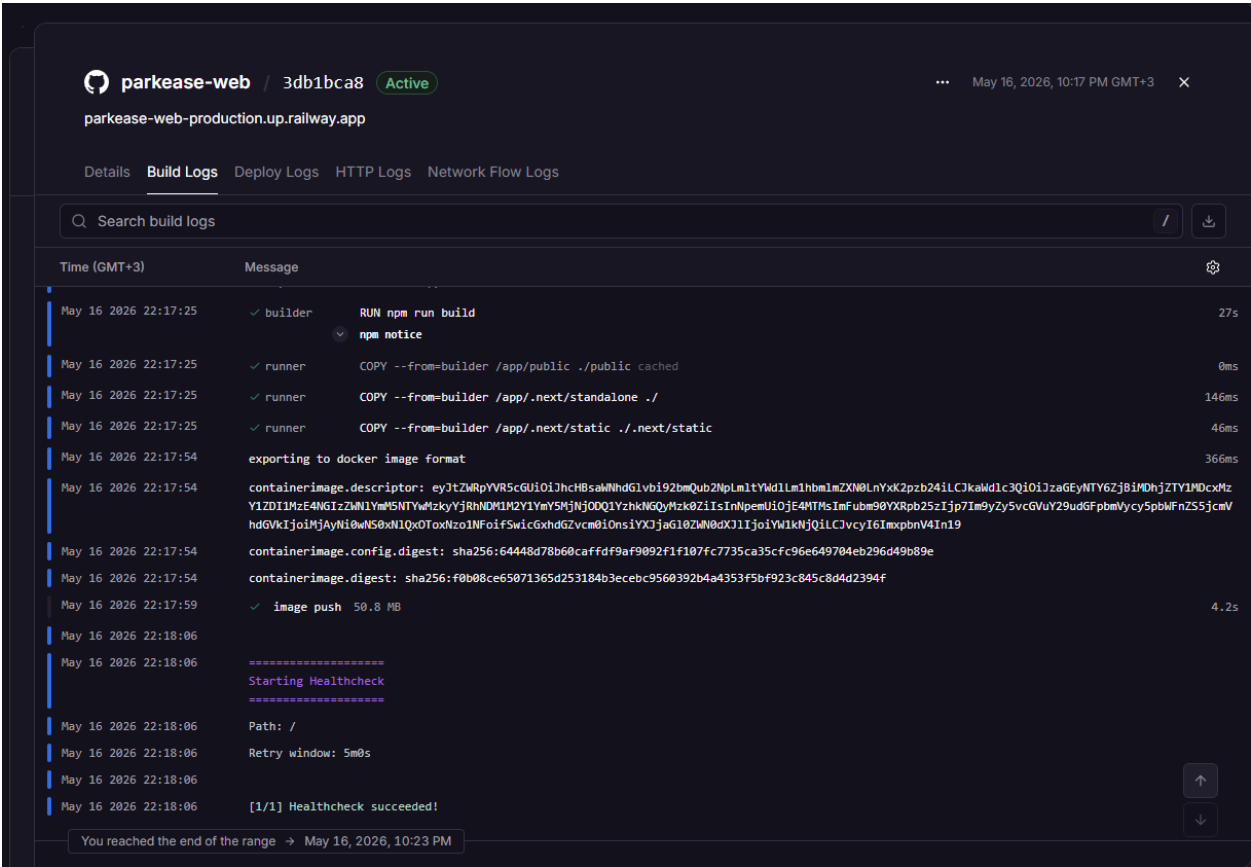


Рисунок 5.5 – Frontend

5.3 Система контроля версий

Исходный код проекта размещён в репозитории GitHub. Разработка велась с использованием стратегии ветвления GitHub Flow: основная ветка `main` содержит стабильный код, новый функционал разрабатывается в `feature`-ветках и вливается через `Pull Request`. Коммиты следуют конвенции `Conventional Commits`.

В процессе разработки были созданы следующие основные ветки: `main` (инициализация), `feat/backend` (схема БД, миграции, `backend`, `REST API`), `feat/frontend` (основные страницы), `feat/testing` (тесты).

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан fullstack клиент-серверный сервис «ParkEase» для поиска и бронирования парковочных мест. Все поставленные задачи были выполнены в полном объёме.

В первой главе проведён анализ предметной области: исследованы текущие процессы поиска парковочных мест (AS-IS), выявлены основные проблемы, выполнен сравнительный анализ трёх существующих решений-аналогов (Яндекс.Парковки, Parkopedia, ParkApp), определены сущности и роли системы, сформулированы функциональные и нефункциональные требования.

Во второй главе спроектирована клиент-серверная архитектура приложения с разделением на фронтенд (Next.js), бэкенд (Go) и базу данных (PostgreSQL). Разработана ER-диаграмма из пяти сущностей, спроектирован REST API с 15 основными эндпоинтами, применена методология двенадцати факторов (12-Factor App). Проектные решения описаны с помощью UML-диаграмм.

В третьей главе описана реализация серверной и клиентской частей приложения. Реализован полный CRUD для всех сущностей, JWT-аутентификация с refresh-токенами, ролевая модель с тремя ролями (user, owner, admin), валидация входных данных.

В четвёртой главе представлены результаты тестирования: unit-тесты бизнес-логики, fuzz-тесты валидации входных данных (5 модулей), тесты ролевой модели на некорректные данные.

В пятой главе описано развёртывание: контейнеризация через Docker с multi-stage сборкой, оркестрация через Docker Compose, автоматический деплой на платформу Railway, использование Git/GitHub с стратегией ветвления GitHub Flow.

Разработанный сервис «ParkEase» представляет собой функционирующее веб-приложение, решающее выявленные проблемы

предметной области и демонстрирующее применение современных технологий и методологий разработки клиент-серверных приложений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Wiggins, A. The Twelve-Factor App [Электронный ресурс]. – URL: <https://12factor.net/> (дата обращения: 10.05.2025).
2. Донован, А. Язык программирования Go / А. Донован, Б. Керниган. – М.: Вильямс, 2021. – 432 с.
3. Go Documentation [Электронный ресурс]. – URL: <https://go.dev/doc/> (дата обращения: 10.05.2025).
4. Next.js Documentation [Электронный ресурс]. – URL: <https://nextjs.org/docs> (дата обращения: 10.05.2025).
5. PostgreSQL 16 Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 10.05.2025).
6. Docker Documentation [Электронный ресурс]. – URL: <https://docs.docker.com/> (дата обращения: 10.05.2025).
7. Leaflet – an open-source JavaScript library for mobile-friendly interactive maps [Электронный ресурс]. – URL: <https://leafletjs.com/> (дата обращения: 10.05.2025).
8. JSON Web Token (JWT) [Электронный ресурс] : RFC 7519 / IETF. – URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 10.05.2025).
9. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017. – 28 с.
10. Tailwind CSS Documentation [Электронный ресурс]. – URL: <https://tailwindcss.com/docs> (дата обращения: 10.05.2025).
11. shadcn/ui – Beautifully designed components [Электронный ресурс]. – URL: <https://ui.shadcn.com/> (дата обращения: 10.05.2025).
12. chi – lightweight, idiomatic and composable router for building Go HTTP services [Электронный ресурс]. – URL: <https://github.com/go-chi/chi> (дата обращения: 10.05.2025).
13. pgx – PostgreSQL driver and toolkit for Go [Электронный ресурс]. – URL: <https://github.com/jackc/pgx> (дата обращения: 10.05.2025).

14. Railway Documentation [Электронный ресурс]. – URL:
<https://docs.railway.app/> (дата обращения: 10.05.2025).

15. Яндекс.Парковки [Электронный ресурс]. – URL:
<https://parking.yandex.ru/> (дата обращения: 10.05.2025).

16. Parkopedia [Электронный ресурс]. – URL:
<https://www.parkopedia.com/> (дата обращения: 10.05.2025).

17. Go Fuzzing [Электронный ресурс]. – URL:
<https://go.dev/doc/security/fuzz/> (дата обращения: 10.05.2025).

ПРИЛОЖЕНИЕ А

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: parking
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d parking"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build:
      context: .
      dockerfile: Dockerfile
    env_file:
      - .env
    environment:
      PORT: 8080
      APP_ENV: development
      DATABASE_URL:
postgres://postgres:postgres@postgres:5432/parking?sslmode=disable
      JWT_SECRET: your-super-secret-key-change-in-production
      JWT_ACCESS_TTL: 15m
      JWT_REFRESH_TTL: 168h
      CORS_ORIGINS: http://localhost:3000
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy

  seed:
    build:
      context: .
      dockerfile: Dockerfile
    command: ["/seed"]
    environment:
      DATABASE_URL:
postgres://postgres:postgres@postgres:5432/parking?sslmode=disable
    depends_on:
      postgres:
        condition: service_healthy
```

```

    backend:
      condition: service_started
profiles:
  - tools

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  env_file:
    - .env
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:8080/api/v1
  ports:
    - "3000:3000"
  depends_on:
    - backend

volumes:
  postgres_data:

```

Листинг A.1 –docker-compose.yml

```

FROM golang:1.26-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o server ./cmd/server
RUN CGO_ENABLED=0 GOOS=linux go build -o seed ./seeds

FROM alpine:3.22
RUN apk --no-cache add ca-certificates
WORKDIR /app
COPY --from=builder /app/server .
COPY --from=builder /app/seed .
COPY migrations/ ./migrations/
EXPOSE 8080
CMD ["/server"]

```

Листинг A.2 –Dockerfile

```

FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .

```

```
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

Листинг A.3 –Dockerfile из папки frontend

ПРИЛОЖЕНИЕ Б

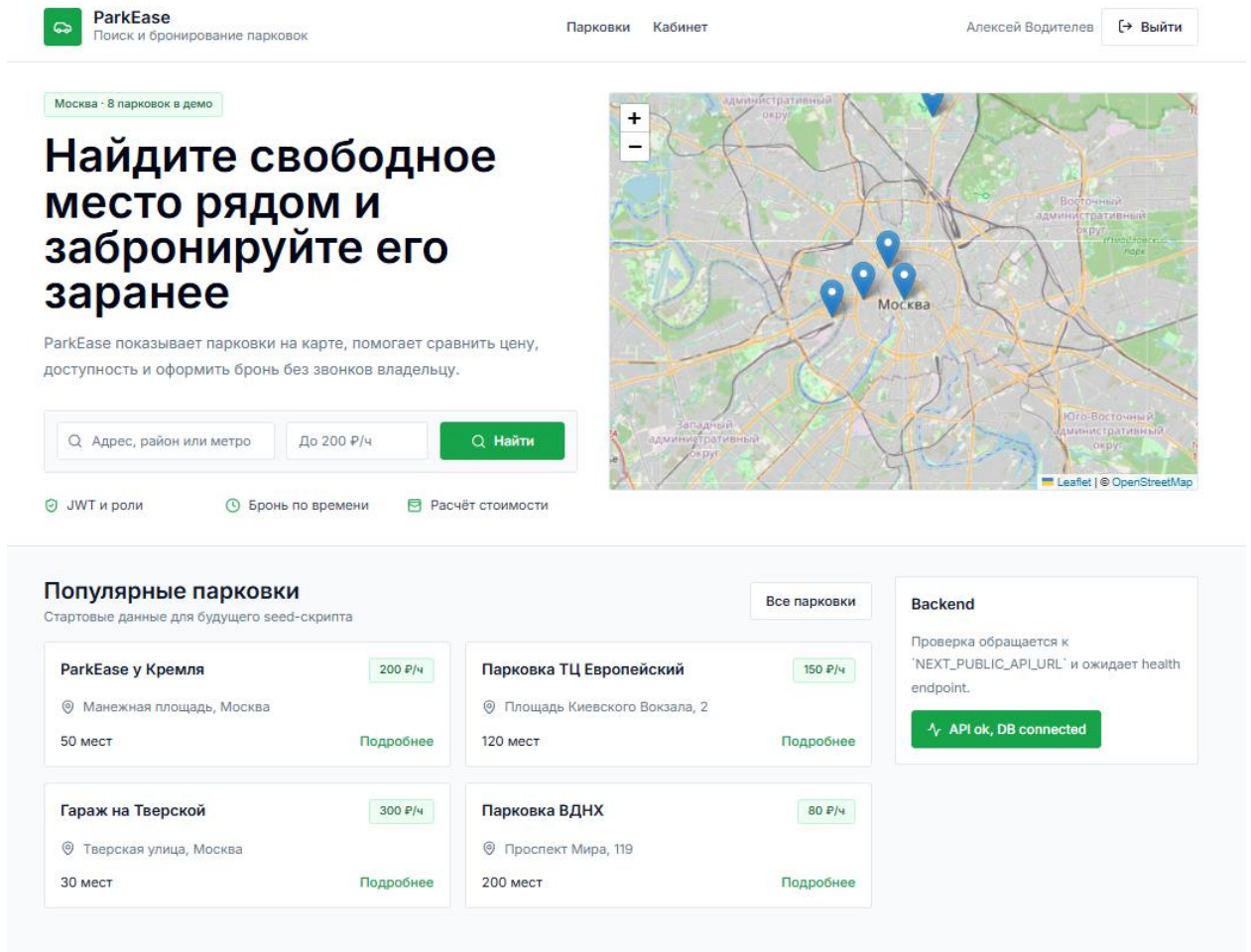


Рисунок Б.1 – Главная страница

Парковки

Поиск по адресу, названию и карте.

ПаркEase у Кремля

200 Р/ч

Манежная площадь, Москва

50 мест

Открыть

Парковка ТЦ Европейский

150 Р/ч

Площадь Киевского Вокзала, 2

120 мест

Открыть

Гараж на Тверской

300 Р/ч

Тверская улица, Москва

30 мест

Открыть

Парковка ВДНХ

80 Р/ч

Проспект Мира, 119

200 мест

Открыть

Подземная у Арбата

250 Р/ч

Новый Арбат, Москва

45 мест

Открыть

Парковка Сити

180 Р/ч

Пресненская набережная, Москва

500 мест

Открыть

Гараж Сокольники

100 Р/ч

Сокольническая площадь, Москва

60 мест

Открыть

Открытая Лужники

120 Р/ч

Лужнецкая набережная, Москва

300 мест

Открыть

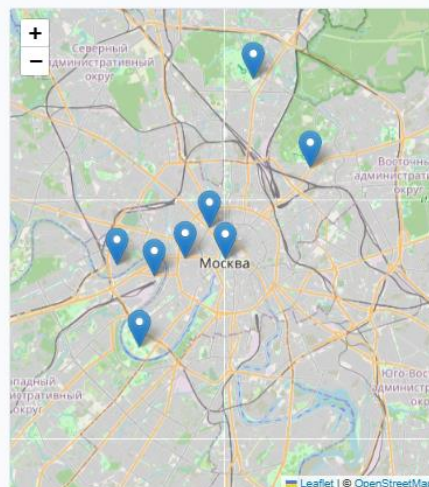


Рисунок Б.2 – Страница “Парковки”

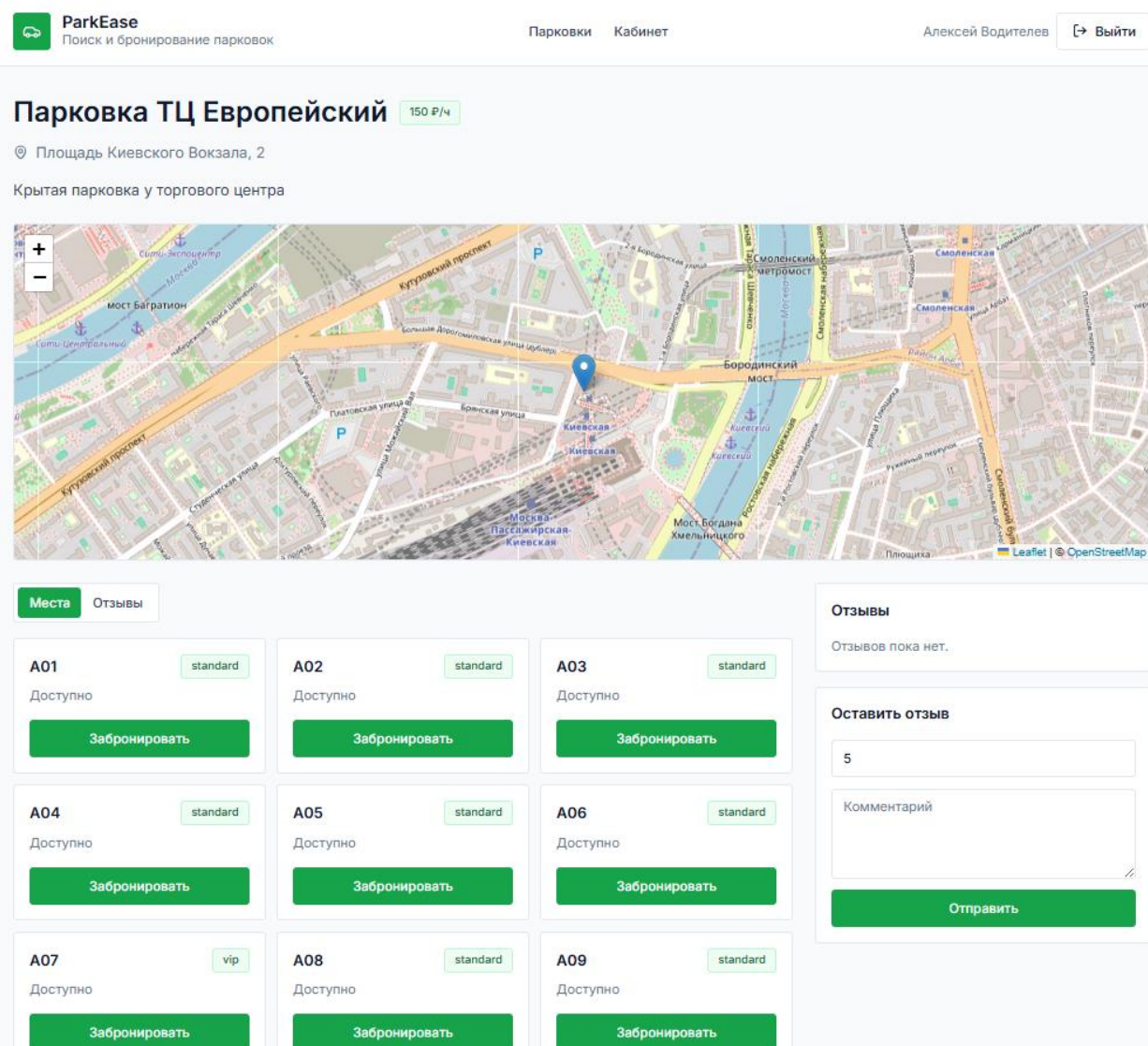


Рисунок Б.3 – Страница парковки “ТЦ Европейский”

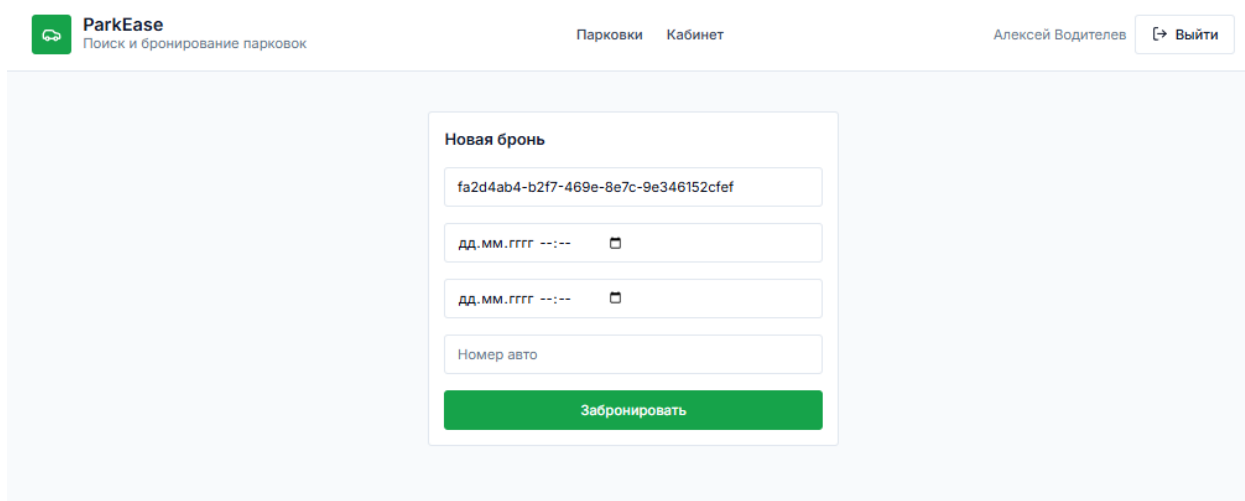


Рисунок Б.4 – Страница брони выбранной парковки

Личный кабинет

user1@parkease.ru

user

Профиль

Алексей Водителев

Телефон

Сохранить

Мои бронирования

Авто	Период	Статус	
A123BC777	17.05.2026, 15:47:00 - 17.05.2026, 17:47:00	cancelled	Отменить

Рисунок Б.5 – Страница личного кабинета